

Module 4 Assignment — Regularization

Module 4 Assignment — Regularization

by

Sri Ram Prabu E

ALY6015: Intermediate Analytics

Dr. Paul Dooley

October 12th, 2025

Module 4 Assignment — Regularization

Introduction

This report investigates whether regularization improves prediction of Graduation Rate (Grad.Rate) across U.S. colleges using the College dataset from the ISLR package. We model Grad.Rate as a function of available institutional features (e.g., enrollment counts, acceptance metrics, expenditures), focusing on two penalized linear models:

- **Ridge regression (L2 penalty)** to shrink correlated coefficients and reduce variance.
- **LASSO regression (L1 penalty)** to both shrink and perform feature selection by zeroing out some coefficients.

We follow a train/test split to estimate out-of-sample error and guard against overfitting, in line with the course's feature-selection handout's recommendation to build and assess models on separate sets (70/30 split example and `createDataPartition` usage).

We evaluate each model using 10-fold cross-validation (via `cv.glmnet`) to select the tuning parameter λ . We record $\lambda_{\min}$ (minimizes cross-validated error) and λ_{1se} (largest λ within one standard error of the minimum). We then compare training and test RMSE to assess fit quality and potential overfitting. Finally, we benchmark penalized models against a stepwise selection baseline as requested in the assignment brief.

Data and Methods

Data Source: ISLR::College (continuous response Grad.Rate).

Pre-processing:

- Remove rows with missing values (the dataset was complete).
- Use `model.matrix` to one-hot encode the Private factor.

Module 4 Assignment — Regularization

- Rely on glmnet's internal standardization (default) to place predictors on comparable scales.

Train/Test Split

- Split the data into 70% train and 30% test using `caret::createDataPartition` with a fixed seed for reproducibility.

Modeling & Tuning

- **Ridge:** `glmnet(..., alpha = 0)` with `cv.glmnet` for λ selection.
- **LASSO:** `glmnet(..., alpha = 1)` with `cv.glmnet`.
- Record $\lambda_{\min}$ and λ_{1se} and interpret the cross-validation plot (MSE vs. $\log(\lambda)$) for each model.

Performance Metrics

- **RMSE (Train)** = $\text{mean}((y_{\text{train}} - \hat{y}^{\text{train}})^2)$
- **RMSE (Test)** = $\text{mean}((y_{\text{test}} - \hat{y}^{\text{test}})^2)$
- Compare train vs. test RMSE for overfitting diagnostics.

Baseline: Stepwise Selection

- Fit a linear model via bidirectional stepwise selection (AIC) as shown in the feature-selection guide, then compute test RMSE for comparison.

Module 4 Assignment — Regularization

Analysis

Ridge Regression ($\alpha=0$)

Cross-Validation Results

- $\lambda_{\min}$ (Ridge): **1.201**
- λ_{1se} (Ridge): **17.832**

Interpretation of the CV plot: The plot shows the mean squared error across 10-fold cross-validation for a range of λ values. The left vertical line marks $\lambda_{\min}$, where the model achieves the lowest average error. The right line marks λ_{1se} , which favors a simpler (more regularized) model whose performance is statistically comparable to the best model. The significant gap between the two lambdas indicates that a much simpler model exists that performs nearly as well.

Coefficients

At $\lambda_{\min}$, all coefficients are shrunk toward zero but none are eliminated. This is a key characteristic of Ridge regression. The shrinkage helps to reduce the impact of multicollinearity among predictors.

Performance

- **lambda.min Model:**
 - Train RMSE (Ridge): **12.67**
 - Test RMSE (Ridge): **12.57**
- **lambda.1se Model:**

Module 4 Assignment — Regularization

- Train RMSE (Ridge): **13.27**
- Test RMSE (Ridge): **12.72**

The test RMSE for the lambda.min model is slightly lower than its training RMSE. While unusual, this can occur by chance in a single data split and strongly indicates that the model is not overfit. The lambda.min model provides the best performance.

LASSO ($\alpha=1$)

Cross-Validation Results

- $\lambda_{\min}$ (LASSO): **0.138**
- λ_{1se} (LASSO): **1.413**

Interpretation: The LASSO CV plot is read in the same way as the Ridge plot. The error curve is similar, but LASSO's underlying mechanism allows it to perform feature selection as it shrinks coefficients.

Coefficients & Feature Selection

At $\lambda_{\min}$, LASSO sets several coefficients exactly to zero, performing automated feature selection. This is its key advantage over Ridge, as it produces a more parsimonious and interpretable model by identifying and removing predictors that contribute little to the model's predictive power.

Performance

- **lambda.min Model:**
 - Train RMSE (LASSO): **12.65**

Module 4 Assignment — Regularization

- Test RMSE (LASSO): **12.72**
- **lambda.1se Model:**
 - Train RMSE (LASSO): **13.26**
 - Test RMSE (LASSO): **12.95**

The LASSO models perform well, with the lambda.min model being competitive. The small gap between training and test RMSE confirms that the models are not overfit.

Stepwise Selection Benchmark

Using the stepwise method reference, we fit a linear model via step() (direction = "both").

- **Selected formula (Stepwise):** Grad.Rate ~ Private + Apps + Top25perc + F.Undergrad + P.Undergrad + Outstate + Room.Board + Personal + perc.alumni + Expend
- Train RMSE (Stepwise): **12.64**
- Test RMSE (Stepwise): **12.94**

The stepwise model achieved the lowest training error, as expected since it is not penalized, but its performance on the test set was weaker than the Ridge model.

Comparison & Discussion

Model	Training RMSE	Test RMSE	Notes
Ridge (lambda.min)	12.67	12.57	Best test performance
LASSO (lambda.min)	12.65	12.72	Competitive performance

Module 4 Assignment — Regularization

Stepwise Regression	12.64	12.94	Weaker test performance
LASSO (lambda.1se)	13.26	12.95	Simpler but less accurate
Ridge (lambda.1se)	13.27	12.72	Simpler, competitive Ridge

Which model performed better and why?

The Ridge regression model using lambda.min performed the best, achieving the lowest Test RMSE of 12.57. This result suggests that the dataset contains many predictors that are genuinely related to graduation rates, and that multicollinearity is likely present. Ridge regression excels in such scenarios by shrinking the coefficients of correlated predictors, reducing model variance without eliminating any variables entirely. This retains the predictive information from all features, leading to superior out-of-sample performance in this case.

Expected vs. Observed:

The prior expectation might have been for LASSO to perform best due to its feature selection capabilities. However, the observed result shows Ridge as the winner. This indicates that the relationships in this dataset are better captured by keeping all predictors and moderating their effects (Ridge) rather than by selecting a smaller subset (LASSO and Stepwise).

Conclusion

This analysis successfully demonstrated the application and value of regularization techniques for predictive modeling.

- The Ridge regression model using lambda.min is the recommended final model. It provided the best predictive accuracy on unseen data, suggesting it best captured the underlying relationships for this problem.
- While Ridge does not eliminate variables, the magnitude of its coefficients still points to the importance of institutional selectivity, resources, and private status in predicting graduation rates.
- The superior performance of Ridge regression implies that a wide range of factors collectively influence college graduation rates. Instead of a few dominant predictors, many variables contribute small but meaningful effects. The model's success highlights the importance of addressing multicollinearity to build a robust predictive model. Ultimately, regularization proved to be a superior approach to the heuristic-based stepwise method for this dataset.

Module 4 Assignment — Regularization

References

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An Introduction to Statistical Learning*. Springer. (ISLR College dataset)
- R Core Team. (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.
- Kuhn, M. (2008). Building predictive models in R using the caret package. *Journal of Statistical Software*.
- Robin, X., et al. (2011). pROC: an open-source package to analyze and compare ROC curves. *BMC Bioinformatics*.
- Friedman, Hastie, Tibshirani (2010). Regularization Paths for Generalized Linear Models via Coordinate Descent (*glmnet*).

Appendix: R

```
set.seed(2025)

suppressPackageStartupMessages({
  library(ISLR)
  library(caret)
  library(glmnet)
  library(dplyr)
})

data("College")

# Inspect and clean
df <- College %>%
  as.data.frame() %>%
  na.omit()

# Response
y <- df$Grad.Rate

# Model matrix (one-hot encode factors, drop intercept -> handled by glmnet)
X <- model.matrix(Grad.Rate ~ . , data = df)[, -1]
```

Module 4 Assignment — Regularization

```
# Train/Test Split (70/30)

set.seed(2025)

idx <- createDataPartition(y, p = 0.7, list = FALSE)

X_train <- X[idx, ]; y_train <- y[idx]

X_test  <- X[-idx,]; y_test  <- y[-idx]


rmse <- function(actual, pred) sqrt(mean((actual - pred)^2))


# Ridge (alpha = 0)

set.seed(2025)

cv_ridge <- cv.glmnet(X_train, y_train, alpha = 0, nfolds = 10, standardize = TRUE)

lambda_min_ridge <- cv_ridge$lambda.min

lambda_1se_ridge <- cv_ridge$lambda.1se


# Coefficients at lambda.min

coef_ridge_min <- coef(cv_ridge, s = "lambda.min")

# Coefficients at lambda.1se (often simpler)

coef_ridge_1se <- coef(cv_ridge, s = "lambda.1se")


# Train/Test predictions & RMSE

pred_ridge_train_min <- predict(cv_ridge, newx = X_train, s = "lambda.min")

pred_ridge_test_min  <- predict(cv_ridge, newx = X_test, s = "lambda.min")
```

Module 4 Assignment — Regularization

```
rmse_ridge_train_min <- rmse(y_train, pred_ridge_train_min)

rmse_ridge_test_min <- rmse(y_test, pred_ridge_test_min)


pred_ridge_train_1se <- predict(cv_ridge, newx = X_train, s = "lambda.1se")
pred_ridge_test_1se <- predict(cv_ridge, newx = X_test, s = "lambda.1se")
rmse_ridge_train_1se <- rmse(y_train, pred_ridge_train_1se)
rmse_ridge_test_1se <- rmse(y_test, pred_ridge_test_1se)


# ---- LASSO (alpha = 1) ----
set.seed(2025)

cv_lasso <- cv.glmnet(X_train, y_train, alpha = 1, nfolds = 10, standardize = TRUE)

lambda_min_lasso <- cv_lasso$lambda.min
lambda_1se_lasso <- cv_lasso$lambda.1se


coef_lasso_min <- coef(cv_lasso, s = "lambda.min")
coef_lasso_1se <- coef(cv_lasso, s = "lambda.1se")


# Identify zeroed coefficients at lambda.min
lasso_min_nonzero <- rownames(coef_lasso_min)[as.numeric(coef_lasso_min) != 0]


pred_lasso_train_min <- predict(cv_lasso, newx = X_train, s = "lambda.min")
pred_lasso_test_min <- predict(cv_lasso, newx = X_test, s = "lambda.min")
rmse_lasso_train_min <- rmse(y_train, pred_lasso_train_min)
```

Module 4 Assignment — Regularization

```
rmse_lasso_test_min <- rmse(y_test, pred_lasso_test_min)

pred_lasso_train_1se <- predict(cv_lasso, newx = X_train, s = "lambda.1se")
pred_lasso_test_1se <- predict(cv_lasso, newx = X_test, s = "lambda.1se")

rmse_lasso_train_1se <- rmse(y_train, pred_lasso_train_1se)
rmse_lasso_test_1se <- rmse(y_test, pred_lasso_test_1se)

# Stepwise baseline
# a full linear model (without regularization)
train_df <- df[idx, ]
test_df <- df[-idx, ]

full_lm <- lm(Grad.Rate ~ ., data = train_df)
step_lm <- step(full_lm, direction = "both", trace = 0)

pred_step_train <- predict(step_lm, newdata = train_df)
pred_step_test <- predict(step_lm, newdata = test_df)
rmse_step_train <- rmse(train_df$Grad.Rate, pred_step_train)
rmse_step_test <- rmse(test_df$Grad.Rate, pred_step_test)

# Summaries to copy into report
cat("\n--- RIDGE  $\lambda$  ---\n")

cat("lambda.min:", lambda_min_ridge, "\n")
```

Module 4 Assignment — Regularization

```
cat("lambda.1se:", lambda_1se_ridge, "\n")

cat("\nRidge RMSE (train/test @  $\lambda_{\min}$ ):", rmse_ridge_train_min, rmse_ridge_test_min, "\n")
cat("Ridge RMSE (train/test @  $\lambda_{1se}$ ):", rmse_ridge_train_1se, rmse_ridge_test_1se, "\n")


cat("\n--- LASSO  $\lambda$  ---\n")

cat("lambda.min:", lambda_min_lasso, "\n")
cat("lambda.1se:", lambda_1se_lasso, "\n")
cat("\nLASSO RMSE (train/test @  $\lambda_{\min}$ ):", rmse_lasso_train_min, rmse_lasso_test_min, "\n")
cat("LASSO RMSE (train/test @  $\lambda_{1se}$ ):", rmse_lasso_train_1se, rmse_lasso_test_1se, "\n")


cat("\n--- Stepwise ---\n")

cat("Formula:", deparse(formula(step_lm)), "\n")
cat("Stepwise RMSE (train/test):", rmse_step_train, rmse_step_test, "\n")


# Optional: CV plots (view in R)
# plot(cv_ridge); title("Ridge CV", line = 2.5)
# plot(cv_lasso); title("LASSO CV", line = 2.5)
```